Introduction to VLSI Design

Homework 1 — Fall 2023

Independent solutions with reviewed reference submissions

Circuit conventions. A bubble marks a PMOS gate. All NMOS bodies connect to GND and all PMOS bodies to V_{DD}. Wires meeting at the shown common nodes are connected. Series means AND of switch-enable conditions; parallel means OR. Device sizes are unrestricted in this logic-design homework.

1(a). Static CMOS for an AOI function

The printed prompt places a bar over the *entire* expression:

$$\boxed{f(w,x,y,z) = \overline{w(x+y)+yz}.}$$

This bar is easy to lose during PDF text extraction. The pull-down network (PDN) must conduct exactly when $\bar{f} = w(x+y)+yz$ is 1. Therefore use

$$[\mathrm{N}(w) \text{ series } (\mathrm{N}(x) \parallel \mathrm{N}(y))] \parallel [\mathrm{N}(y) \text{ series } \mathrm{N}(z)].$$

Replacing every series connection by parallel and every parallel connection by series gives the complementary pull-up network (PUN):

$$[\mathrm{P}(w) \parallel (\mathrm{P}(x) \text{ series } \mathrm{P}(y))] \text{ series } [\mathrm{P}(y) \parallel \mathrm{P}(z)].$$

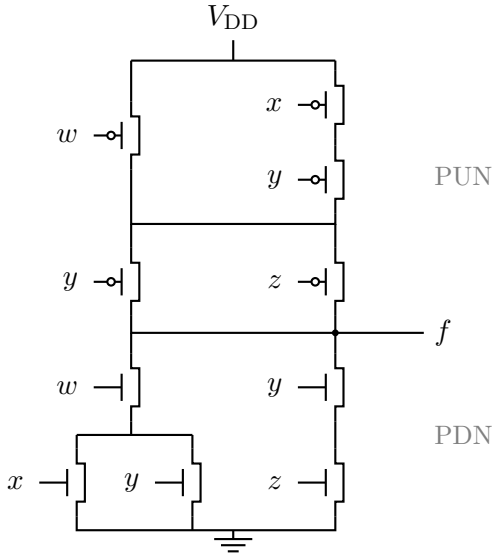


Figure 1. The requested AOI gate: 5 NMOS + 5 PMOS.

The PUN conducts when $(\bar{w}+\bar{x}\bar{y})(\bar{y}+\bar{z}) = f$. The two networks are complementary for all 16 input combinations. The output is already f; an extra inverter would produce the wrong polarity.

1(b). Static CMOS for the minority function

The requested function is

$$f(x, y, z) = \bar{x}\bar{y} + \bar{x}\bar{z} + \bar{y}\bar{z}.$$

It is 1 exactly when at most one of x, y, z is 1. Its complement is the three-input majority function:

$$\bar{f} = xy + xz + yz = x(y + z) + yz.$$

The second equality shares the transistor controlled by x. A factored 10-transistor implementation uses

PDN: [N(x) series (N(y) ‖ N(z))] ‖ [N(y) series N(z)],
PUN: [P(x) ‖ (P(y) series P(z))] series [P(y) ‖ P(z)].

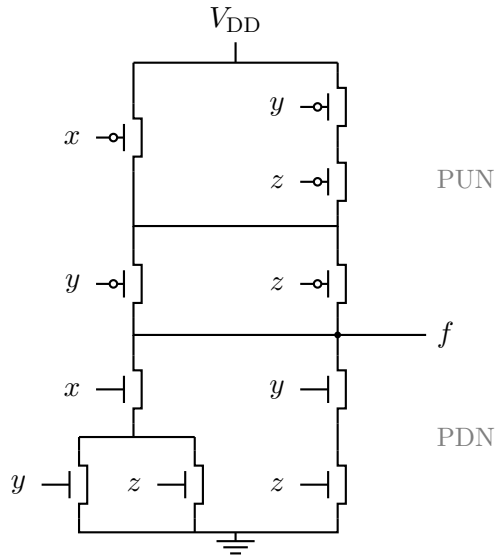


Figure 2. The minority gate: 5 NMOS + 5 PMOS.

No complemented input wires are needed: a PMOS controlled by x turns on when $x = 0$. Thus both parts of Question 1 use only true input literals, as the prompt requires.

x	y	z	f	x	y	z	f
0	0	0	1	1	0	0	1
0	0	1	1	1	0	1	0
0	1	0	1	1	1	0	0
0	1	1	0	1	1	1	0

2. Four-output controlled rotation

Each legal control word has exactly one 0. Use 4 CMOS transmission gates (TGs) at each output, one for each possible data source. If $c_i = 0$, turn on the corresponding TG using NMOS gate $\bar{c}_i$ and PMOS gate c_i.

Reading the original table in the stated order c_3, c_2, c_1, c_0 gives

$$\begin{aligned} z_3 &= \bar{c}_3 a_3 + \bar{c}_2 a_2 + \bar{c}_1 a_1 + \bar{c}_0 a_0, \\ z_2 &= \bar{c}_3 a_2 + \bar{c}_2 a_1 + \bar{c}_1 a_0 + \bar{c}_0 a_3, \\ z_1 &= \bar{c}_3 a_1 + \bar{c}_2 a_0 + \bar{c}_1 a_3 + \bar{c}_0 a_2, \\ z_0 &= \bar{c}_3 a_0 + \bar{c}_2 a_3 + \bar{c}_1 a_2 + \bar{c}_0 a_1. \end{aligned}$$

These equations describe the valid control words. In compact form, $z_j = \sum_{k=0}^{3} \bar{c}_{3-k} a_{(j-k) \bmod 4}$.

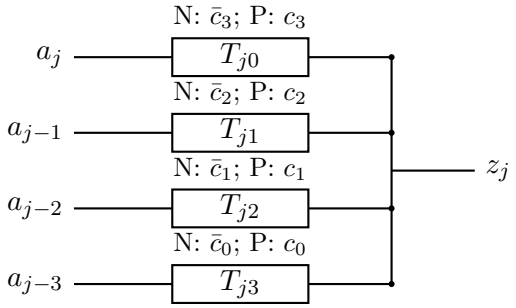
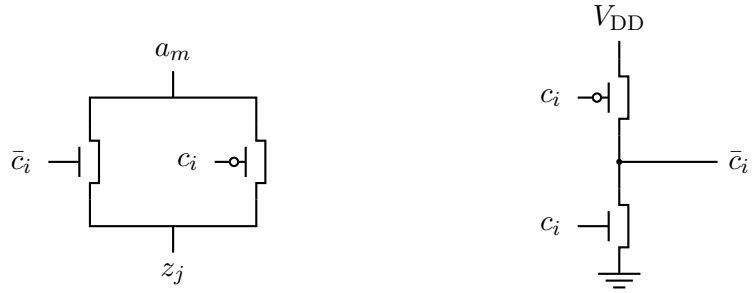


Figure 3. One output bank; build four copies for $j = 0, 1, 2, 3$.

All data subscripts in Figure 3 are modulo 4. The following table lists every data-to-output connection, so the diagram specifies all 16 TGs:

Output	$c_3 = 0$	$c_2 = 0$	$c_1 = 0$	$c_0 = 0$
z_3	a_3	a_2	a_1	a_0
z_2	a_2	a_1	a_0	a_3
z_1	a_1	a_0	a_3	a_2
z_0	a_0	a_3	a_2	a_1

2. Transmission-gate transistor implementation



One TG: parallel NMOS and PMOS CMOS inverter: 2 transistors

Figure 4. Expanded TG and its complementary-control generator.

For each $i = 0, 1, 2, 3$, generate $\bar{c}_i$ with one shared CMOS inverter. When $c_i = 0$, both TG transistors conduct. When $c_i = 1$, both are off. The NMOS passes a strong 0 and the PMOS passes a strong 1, giving a full-swing static connection to the selected data input.

The total is

$$4 \text{ outputs} \times 4 \text{ TGs} \times 2 \text{ MOSFETs} + 4 \text{ inverters} \times 2 \text{ MOSFETs} = \boxed{40 \text{ transistors.}}$$

The four control inverters serve all output banks. Under a legal control word, each output is connected to exactly one data input. No weak NMOS-only pass-high path and no data-to-data contention exists in these settled legal states.

Control-domain assumption. The problem specifies only 0111, 1011, 1101 and 1110. This design assumes those stable one-low words. For 1111 the outputs float; with several controls low, several data inputs may be shorted together. These omitted control cases are not assigned an output value by the prompt. A separate decoder would be required if operation outside the four specified states were also required. The verification script characterizes all omitted control combinations without claiming that they satisfy an unstated specification.

Reproducible connectivity. In `circuits.json`, `TjkN` and `TjkP` connect $a_{(j-k) \bmod 4}$ to z_j. Their gates are respectively $\bar{c}_{3-k}$ and c_{3-k}, exactly as in Figures 3 and 4. The four inverter pairs are also listed explicitly.

3(a). Function of the pass-transistor circuit

The upper NMOS is enabled by A and passes $\bar{B}$. The lower NMOS is enabled by $\bar{A}$ and passes B. Thus

$$F = \begin{cases} B, & A = 0, \\ \bar{B}, & A = 1, \end{cases} \qquad \boxed{F = A\bar{B} + \bar{A}B = A \oplus B.}$$

A	B	F	Enabled path
0	0	0	Lower transistor passes B
0	1	1	Lower transistor passes B
1	0	1	Upper transistor passes $\bar{B}$
1	1	0	Upper transistor passes $\bar{B}$

This Boolean interpretation does not claim that an NMOS alone passes an ideal rail-level 1; its high voltage can be degraded by the threshold voltage.

3(b). Full-swing static CMOS XOR

For a complementary static CMOS replacement, $\bar{F} = AB + \bar{A}\bar{B}$. Put these two two-transistor series branches in parallel in the PDN. The dual PUN consists of two parallel pairs in series. Generate $\bar{A}$ and $\bar{B}$ with two copies of the CMOS inverter in Figure 4.

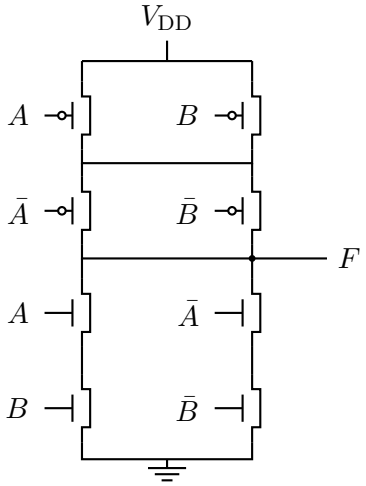


Figure 5. XOR core: 4 NMOS + 4 PMOS, plus two input inverters.

PUN enable is $(\bar{A} + \bar{B})(A + B) = A\bar{B} + \bar{A}B$. Total transistor count, including the input inverters, is $8 + 4 = \boxed{12}$.

Verification and comparison with supplied answers

Run `python check_logic.py`. The checker evaluates independently specified Boolean formulas and table entries, then determines actual outputs by graph reachability through enabled MOS switches. It does not substitute the expected output into a network model.

Circuit	Input cases	Output checks
Question 1(a)	16	16
Question 1(b)	8	8
Question 3 (including input inverters)	4	4
Question 2 (4 legal controls × 16 data words)	64	256
Total		284

Result: all 284 functional output checks pass. For each static CMOS case the output reaches exactly one supply rail, so there is neither a floating output nor a direct rail short in the ideal-switch model. Every legal TG output reaches exactly the required data input. The script also records 768 output characterizations across the omitted control cases. This homework contains no SPICE timing or sizing requirement; these checks do not establish analog voltage margins, delay or power.

Supplied reference	Review and final integration
109501570, HW1	Correct AOI polarity and XOR. Its 12-transistor minority gate is valid; the factored 10-transistor network is used here. Its encoded two-level TG rotation is replaced by the directly controlled bank.
110501512, HW1	Its Q1 transistor networks and directly controlled TG bank provide useful topology checks. The minority-function algebra is rewritten from the prompt. Its XOR core assumes complements; the final solution adds explicit CMOS complement generators.
110501558, HW1	Its photographed Q1(a) includes an extra output inverter, which implements the positive AO expression rather than the printed AOI function. Its minority and XOR ideas are consistent with the requested functions; the final diagrams use the factored network and explicit complement generators.

All three supplied submissions were reviewed visually, including handwritten figures. The final equations, diagrams and connectivity model were derived independently and cross-checked against those submissions. The original assignment is preserved at 2023/assignments/homework1.pdf. Full reference filenames and source hashes appear in `REFERENCE_COMPARISON.md`.

Generated evidence. `truth_tables.csv` contains all 28 logic rows; `mux_connectivity.csv` contains the 256 legal output checks; `illegal_controls.csv` records the unspecified control cases; `verification.json` summarizes the results; `circuits.json` lists every checked transistor connection.